

COMPLETE ROADMAP & COURSEWORK

React

From DOM manipulation to modern component architecture

The full story: why React exists, how it thinks, hooks, patterns, state management, and the full ecosystem

7 Phases · Roadmap + Full Coursework with Code · Complete Reference

7 PHASES	50+ CONCEPTS	20 PROJECTS	~5mo TIMELINE	1 MENTAL MODEL TO MASTER
-------------	-----------------	----------------	------------------	-----------------------------

PHASE 0 The Story Before React

Why the jQuery era broke down and what React changed

0.1 A Brief History of Frontend Development

Before React, building interactive web UIs was a discipline of pain. Understanding why helps you understand every design decision React made.

Era	Approach	The Problem
Static HTML (pre-2005)	HTML files, forms, full-page reloads	No interactivity — every click reloads the page
jQuery era (2006–2013)	Direct DOM manipulation with jQuery	Spaghetti: UI state scattered across DOM — impossible to reason about at scale
MVC Frameworks (2010–2013)	Backbone, Knockout, early Angular	Better structure but two-way data binding caused unpredictable cascading updates
React (2013–present)	Declarative components, one-way data flow, virtual DOM	Predictable: describe what the UI should look like; React figures out how to update the DOM
Hooks Era (2019–present)	Function components with hooks replace class components	Simpler: share stateful logic without complex class hierarchies or render props

React's Core Insight

Instead of asking 'how do I change the DOM when something happens?', React asks 'what should the UI look like given this state?'. This shift from imperative to declarative is the single most important thing to understand about React.

0.2 The Roadmap at a Glance

Phase 1 — JS Prerequisites

Closures, this, ES6+, modules, array methods, async/await. React needs these fluently.

Phase 2 — React Fundamentals

JSX, components, props, state, events, conditional rendering, lists.

Phase 3 — Hooks In Depth

useState, useEffect, useRef, useContext, useMemo, useCallback, custom hooks.

Phase 4 — Component Patterns

Composition, lifting state, HOCs, render props, compound components.

Phase 5 — State Management Context API, Redux Toolkit, Zustand, React Query / TanStack Query.	Phase 6 — Performance React.memo, lazy/Suspense, code splitting, profiler, avoiding re-renders.
Phase 7 — Ecosystem React Router, Next.js, TypeScript with React, testing, deployment.	

PHASE 1 JavaScript Prerequisites

The JS you need fluent before React makes sense

1.1 ES6+ Syntax React Relies On

React code is dense with modern JavaScript. These patterns appear in almost every React component.

```
// Destructuring — used everywhere in React
const { name, age, role = 'user' } = person; // object, with default
const [first, ...rest] = items;              // array destructuring
const { address: { city } } = user;          // nested

// Spread — merging objects and arrays (used in state updates)
const updated = { ...user, name: 'Bob' };    // shallow copy + override
const combined = [...list, newItem];         // append without mutation

// Template literals
const greeting = `Hello, ${user.name}!`;

// Short-circuit evaluation — used for conditional rendering
const el = isLoggedIn && <Dashboard />;      // renders Dashboard or false
const val = data ?? 'Loading...';           // nullish coalescing

// Optional chaining — safe property access
const city = user?.address?.city;           // undefined, not an error

// Arrow functions preserve 'this' context
const double = arr.map(x => x * 2);
const evens = arr.filter(x => x % 2 === 0);
const sum = arr.reduce((acc, x) => acc + x, 0);

// Dynamic import — used for code splitting
const Component = React.lazy(() => import('./HeavyComponent'));
```

1.2 Closures — How React Hooks Work

React hooks are built on closures. A hook like `useState` captures the state value in a closure that persists between renders. Without understanding closures, hooks will feel like magic.

```
// A closure captures variables from its outer scope
function makeAdder(x) {
  return (y) => x + y; // inner function 'closes over' x
}
```

```
}
const add5 = makeAdder(5);
add5(3); // 8 - x=5 is remembered

// useState works similarly - React maintains a 'cell' for each hook
// On each render, the hook function re-runs but reads from the same cell

// The stale closure problem (very common React bug):
function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      // BUG: count is captured from first render - always 0
      setCount(count + 1);
    }, 1000);
    return () => clearInterval(id);
  }, []); // empty deps = closure captured count=0 forever

  // FIX: use the functional updater form - no closure needed
  useEffect(() => {
    const id = setInterval(() => {
      setCount(prev => prev + 1); // always reads fresh value
    }, 1000);
    return () => clearInterval(id);
  }, []);
}
```

1.3 Promises & async/await

```
// React often fetches data - you need async patterns fluent

// Fetch with async/await
async function fetchUser(id) {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

// Promise.all - fetch multiple resources in parallel
const [user, posts] = await Promise.all([
  fetchUser(1),
  fetchPosts(1),
]);

// Error handling
try {
  const data = await fetchUser(id);
} catch (err) {
  console.error('Failed:', err.message);
}
```

Phase 1 Projects



ES6 Utility Belt

Build a small utility library using only modern JS: a debounce function, a deep clone, a group-by, a memoizer. No React yet — pure JS fluency.

Concepts: closures · arrow functions · destructuring · spread · modules

PHASE 2 React Fundamentals*JSX, components, props, state, and the React model*

2.1 JSX — Not HTML

JSX looks like HTML but it is JavaScript. It compiles to `React.createElement()` calls. Understanding this removes all the mystery around JSX rules.

```
// JSX compiles to:
const el = <h1 className="title">Hello</h1>;
// React.createElement('h1', { className: 'title' }, 'Hello')

// JSX rules that differ from HTML:
// 1. Use className, not class
// 2. Use htmlFor, not for
// 3. Self-close void elements: <br /> <img />
// 4. Must have a single root element (or Fragment)
// 5. JavaScript expressions in {curly braces}

function Card({ title, count, active }) {
  return (
    <>
      <h2 className={active ? 'active' : 'inactive'}>{title}</h2>
      <p>Count: {count * 2}</p>
      <p>{count > 10 ? 'High' : 'Low'}</p>
      {active && <span>Active!</span>}
    </>
  );
}

// Expressions can be: strings, numbers, JSX, arrays, null/undefined/boolean
// Objects cannot be rendered directly – wrap in JSON.stringify() or .toString()
```

2.2 Components, Props, and the Component Tree

A React application is a tree of components. Data flows down through props. Events flow up through callback props. This one-way flow is what makes React predictable.

```
// Function component – the only kind you should write today
function Button({ label, onClick, disabled = false, variant = 'primary' }) {
  return (
    <button
```

```
        className={`btn btn-${variant}`}
        onClick={onClick}
        disabled={disabled}
      >
        {label}
      </button>
    );
  }

// Children prop - compose components
function Card({ children, title }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-body">{children}</div>
    </div>
  );
}

// Usage: children are passed as JSX between the opening and closing tags
function App() {
  return (
    <Card title="Welcome">
      <p>This is the card body.</p>
      <Button label="Click Me" onClick={() => alert('hi')} />
    </Card>
  );
}

// Props are read-only - NEVER mutate props
// A component is a pure function of its props + state
```

2.3 State with useState

State is data that a component owns and that, when changed, causes the component to re-render. Rules: never mutate state directly, always use the setter function.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // initial value = 0

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```



```
// Functional update - use when new state depends on old state
setCount(prev => prev + 1); // always correct
setCount(count + 1);       // may be stale in async contexts

// State with objects - must spread to avoid mutating
const [user, setUser] = useState({ name: 'Alice', age: 30 });

// WRONG - mutates state directly
user.name = 'Bob'; setUser(user);

// CORRECT - create new object
setUser({ ...user, name: 'Bob' });

// State with arrays - never use .push(), .splice(), etc.
const [items, setItems] = useState([]);
setItems([...items, newItem]); // add
setItems(items.filter(i => i !== item)); // remove
setItems(items.map(i => i.id === id ? {...i, done: true} : i)); // update
```

2.4 Conditional Rendering & Lists

```
// Conditional rendering patterns
function Status({ isLoading, error, data }) {
  if (isLoading) return <Spinner />;
  if (error)      return <Error message={error.message} />;
  if (!data)      return null; // render nothing
  return <DataView data={data} />;
}

// Inline conditional with &&
{user.isAdmin && <AdminPanel />}

// Ternary operator
{isLoggedIn ? <Dashboard /> : <Login />}

// Rendering lists - key prop is required
function List({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        // key must be unique and stable (not index if list can reorder)
        <li key={todo.id}>
          <span>{todo.text}</span>
          {todo.done && <span>✓</span>}
        </li>
      ))}
    </ul>
  );
}
```

Phase 2 Projects



Todo App

Classic but essential. Components: `TodoInput`, `TodoList`, `TodoItem`. State: array of todos. Features: add, toggle done, delete, filter by status. Use `useState` only — no external libraries.

Concepts: [useState](#) · [props](#) · [conditional rendering](#) · [list rendering](#) · [lifting state](#)



Weather Dashboard

Fetch weather from a public API (Open-Meteo is free). Show current conditions and a 5-day forecast. Handle loading, error, and empty states with conditional rendering.

Concepts: [fetch](#) · [conditional rendering](#) · [component composition](#) · [props](#)

PHASE 3 Hooks In Depth

The complete hooks system — built-in and custom

3.1 useEffect — Side Effects & Lifecycle

`useEffect` runs after the component renders. It is where you fetch data, subscribe to events, interact with the DOM, and set up timers. The dependency array controls when it re-runs.

```
import { useState, useEffect } from 'react';

// 1. No dependency array - runs after EVERY render
useEffect(() => { document.title = `Count: ${count}`; });

// 2. Empty array [] - runs ONCE after first render (componentDidMount)
useEffect(() => {
  fetchData().then(setData);
}, []);

// 3. With dependencies - runs when any dep changes
useEffect(() => {
  fetchUser(userId).then(setUser);
}, [userId]); // re-fetch whenever userId changes

// 4. Cleanup function - runs before next effect and on unmount
useEffect(() => {
  const controller = new AbortController();

  fetch(`/api/users/${id}`, { signal: controller.signal })
    .then(r => r.json())
    .then(setUser)
    .catch(err => { if (err.name !== 'AbortError') setError(err); });

  return () => controller.abort(); // cleanup: cancel request if component
  unmounts
}, [id]);

// Common useEffect patterns:
// - Data fetching (see React Query for better DX)
// - Event listener: document.addEventListener / removeEventListener
// - WebSocket: open / close connection
// - Subscription: observable.subscribe / unsubscribe
// - DOM measurement: ref.current.getBoundingClientRect()
```

The Rule of Hooks

Hooks must be called at the top level of a function component — never inside loops, conditions, or nested functions. React relies on the order of hook calls being the same on every render to correctly associate state with hooks.

3.2 useRef — Mutable Values Without Re-renders

```
import { useRef, useEffect } from 'react';

// Use case 1: Access DOM elements directly
function AutoFocusInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus(); // direct DOM access
  }, []);

  return <input ref={inputRef} />;
}

// Use case 2: Store mutable values that don't trigger re-renders
function Stopwatch() {
  const [elapsed, setElapsed] = useState(0);
  const intervalRef = useRef(null); // stores the interval ID

  const start = () => {
    intervalRef.current = setInterval(() => {
      setElapsed(e => e + 1);
    }, 100);
  };

  const stop = () => clearInterval(intervalRef.current);

  return (
    <div>
      <p>{(elapsed / 10).toFixed(1)}s</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}

// Use case 3: Track previous value
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current; // returns value from PREVIOUS render
}
```

3.3 useContext — Sharing State Without Prop Drilling

```
import { createContext, useContext, useState } from 'react';
```

```
// Step 1: Create the context with a default value
const ThemeContext = createContext({ theme: 'light', toggleTheme: () => {} });

// Step 2: Provide the context near the root of your app
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme(t => t === 'light' ? 'dark' : 'light');

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// Step 3: Consume the context in any descendant component
function Header() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  return (
    <header className={`header-${theme}`}>
      <button onClick={toggleTheme}>
        Switch to {theme === 'light' ? 'dark' : 'light'}
      </button>
    </header>
  );
}

// Step 4: Wrap the app
function App() {
  return (
    <ThemeProvider>
      <Header />
      <main>...</main>
    </ThemeProvider>
  );
}
```

3.4 useMemo & useCallback — Performance Hooks

```
import { useMemo, useCallback } from 'react';

// useMemo - memoize an expensive computed value
// Only recomputes when dependencies change
function ProductList({ products, filterText }) {
  const filtered = useMemo(
    () => products.filter(p => p.name.includes(filterText)),
    [products, filterText] // only recompute when these change
  );

  return <ul>{filtered.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}
```

```

}

// useCallback — memoize a function reference
// Prevents child component re-renders when the parent re-renders
function Parent() {
  const [count, setCount] = useState(0);

  // Without useCallback: new function reference every render → Child re-renders
  // With useCallback: stable reference → Child only re-renders when userId
  changes
  const handleUserUpdate = useCallback((userId, data) => {
    updateUser(userId, data);
  }, []); // no deps — function never needs to change

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(c => c + 1)}>+</button>
      <UserForm onSubmit={handleUserUpdate} />
    </div>
  );
}

// RULE: Don't use useMemo/useCallback everywhere.
// Add them only when you measure a performance problem.
// Premature memoisation adds complexity without benefit.

```

3.5 Custom Hooks — Reusable Stateful Logic

Custom hooks let you extract component logic into reusable functions. If you find yourself duplicating `useEffect` + `useState` patterns, extract them into a custom hook.

```

// useFetch — reusable data fetching hook
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    setError(null);

    fetch(url)
      .then(r => r.json())
      .then(data => { if (!cancelled) setData(data); })
      .catch(err => { if (!cancelled) setError(err); })
      .finally(() => { if (!cancelled) setLoading(false); });

    return () => { cancelled = true; };
  }, [url]);
}

```

```
    return { data, loading, error };
  }

  // Usage — one line instead of 15
  function UserProfile({ id }) {
    const { data: user, loading, error } = useFetch(`/api/users/${id}`);
    if (loading) return <Spinner />;
    if (error) return <Error message={error.message} />;
    return <div>{user.name}</div>;
  }

  // useLocalStorage — persist state to localStorage
  function useLocalStorage(key, initialValue) {
    const [value, setValue] = useState(() => {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initialValue;
    });

    const setAndStore = (newValue) => {
      setValue(newValue);
      localStorage.setItem(key, JSON.stringify(newValue));
    };

    return [value, setAndStore];
  }

  // useDebounce — delay a value update
  function useDebounce(value, delay) {
    const [debounced, setDebounced] = useState(value);
    useEffect(() => {
      const timer = setTimeout(() => setDebounced(value), delay);
      return () => clearTimeout(timer);
    }, [value, delay]);
    return debounced;
  }
}
```

Phase 3 Projects



Search with Debounce

Search component that queries an API as the user types, but debounced (wait 300ms after last keystroke). Built entirely with custom hooks: useDebounce, useFetch.

Concepts: [useEffect](#) · [useRef](#) · [custom hooks](#) · [debounce](#) · [async](#)



Theme + Auth Context

Build a full context system: ThemeContext (light/dark), AuthContext (user session). Persist to localStorage. Show how context avoids prop drilling through 5 levels of components.

Concepts: [useContext](#) · [createContext](#) · [useEffect](#) · [custom hooks](#) · [localStorage](#)

PHASE 4 Component Patterns

Composition, HOCs, compound components, and Portals

4.1 Composition Over Inheritance

```
// BAD: inheritance (don't do this in React)
class FancyButton extends Button { /* ... */ }

// GOOD: composition — combine components using children and props

// Specialisation via composition:
function PrimaryButton(props) {
  return <Button {...props} variant="primary" size="lg" />;
}

// Container / Presentational pattern:
// Presentational — only cares about appearance
function UserCard({ name, avatar, bio, onFollow }) {
  return (
    <div className="user-card">
      <img src={avatar} alt={name} />
      <h3>{name}</h3>
      <p>{bio}</p>
      <button onClick={onFollow}>Follow</button>
    </div>
  );
}

// Container — handles data and logic
function UserCardContainer({ userId }) {
  const { data: user, loading } = useFetch(`/api/users/${userId}`);
  if (loading) return <Skeleton />;
  return <UserCard {...user} onFollow={() => followUser(userId)} />;
}
```

4.2 Compound Components

Compound components share implicit state through context, creating an API that feels natural and composable. Used by Select, Tabs, Accordion, and Modal components.

```
import { createContext, useContext, useState } from 'react';

const TabsContext = createContext(null);

// Parent provides context
```



```
function Tabs({ children, defaultTab }) {
  const [active, setActive] = useState(defaultTab);
  return (
    <TabsContext.Provider value={{ active, setActive }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

// Children consume context implicitly
Tabs.List = function TabList({ children }) {
  return <div className="tab-list" role="tablist">{children}</div>;
};

Tabs.Tab = function Tab({ id, children }) {
  const { active, setActive } = useContext(TabsContext);
  return (
    <button
      role="tab"
      aria-selected={active === id}
      className={active === id ? 'tab active' : 'tab'}
      onClick={() => setActive(id)}
    >
      {children}
    </button>
  );
};

Tabs.Panel = function TabPanel({ id, children }) {
  const { active } = useContext(TabsContext);
  if (active !== id) return null;
  return <div role="tabpanel">{children}</div>;
};

// Usage — clean, reads like HTML
<Tabs defaultTab="profile">
  <Tabs.List>
    <Tabs.Tab id="profile">Profile</Tabs.Tab>
    <Tabs.Tab id="settings">Settings</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel id="profile"><ProfilePanel /></Tabs.Panel>
  <Tabs.Panel id="settings"><SettingsPanel /></Tabs.Panel>
</Tabs>
```

4.3 useReducer — Complex State Logic

```
import { useReducer } from 'react';

// useReducer is better than useState when:
// - Next state depends on previous state
// - Multiple sub-values are updated together
// - State transitions have names (actions)

const initialState = { items: [], status: 'idle', error: null };
```

```
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      return { ...state, items: [...state.items, action.payload] };
    case 'REMOVE_ITEM':
      return { ...state, items: state.items.filter(i => i.id !== action.payload) };
    case 'UPDATE_QTY':
      return { ...state, items: state.items.map(i =>
        i.id === action.payload.id ? { ...i, qty: action.payload.qty } : i
      ) };
    case 'CLEAR_CART':
      return { ...initialState };
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function Cart() {
  const [state, dispatch] = useReducer(cartReducer, initialState);

  const addItem = (item) => dispatch({ type: 'ADD_ITEM', payload: item });
  const removeItem = (id) => dispatch({ type: 'REMOVE_ITEM', payload: id });

  return (
    <div>
      {state.items.map(item => (
        <CartItem key={item.id} {...item} onRemove={() => removeItem(item.id)} />
      ))}
    </div>
  );
}
```

Phase 4 Projects



Compound Component Design System

Build a small design system: Tabs, Accordion, Modal, and Dropdown — all using the compound component pattern. Each one shares state via context internally.

Concepts: [compound components](#) · [createContext](#) · [useReducer](#) · [portals](#) · [accessibility](#)

PHASE 5 State Management

Context, Redux Toolkit, Zustand, and React Query

5.1 When to Use What

Solution	Best For	Not Ideal For
useState	Local UI state (open/closed, form values, toggles)	Shared state across distant components
useContext	Low-frequency global state (theme, auth, locale)	High-frequency updates — causes all consumers to re-render
Redux Toolkit	Large apps, complex state logic, time-travel debugging	Simple apps — significant boilerplate
Zustand	Medium apps, minimal boilerplate, flexible	Apps needing Redux DevTools integration (though Zustand has it too)
React Query / TanStack	Server state: fetching, caching, syncing remote data	Client-only state (UI state, form state)

5.2 Redux Toolkit — Modern Redux

```
// npm install @reduxjs/toolkit react-redux
import { createSlice, configureStore } from '@reduxjs/toolkit';
import { Provider, useSelector, useDispatch } from 'react-redux';

// createSlice generates actions and reducers together
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: state => { state.value += 1; }, // Immer: mutate draft safely
    decrement: state => { state.value -= 1; },
    incrementByAmount: (state, action) => { state.value += action.payload; },
  }
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;

// Configure the store
const store = configureStore({
  reducer: { counter: counterSlice.reducer }
});

// Wrap app with Provider
```

```
function App() { return <Provider store={store}><Counter /></Provider>; }

// Use in components
function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();
  return (
    <div>
      <p>{count}</p>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
    </div>
  );
}
```

5.3 Zustand — Minimal State Management

```
// npm install zustand
import { create } from 'zustand';

// Create a store — no Provider needed
const useCartStore = create((set, get) => ({
  items: [],
  total: 0,

  addItem: (item) => set(state => ({
    items: [...state.items, item],
    total: state.total + item.price,
  })),

  removeItem: (id) => set(state => {
    const item = state.items.find(i => i.id === id);
    return {
      items: state.items.filter(i => i.id !== id),
      total: state.total - (item?.price ?? 0),
    };
  })),

  clearCart: () => set({ items: [], total: 0 }),
}));

// Use anywhere — no Provider, no boilerplate
function CartIcon() {
  const count = useCartStore(state => state.items.length); // subscribe to slice
  return <span>{count}</span>;
}

function CartPage() {
  const { items, total, removeItem, clearCart } = useCartStore();
  // ...
}
```

5.4 TanStack Query — Server State

```
// npm install @tanstack/react-query
import { QueryClient, QueryClientProvider, useQuery, useMutation } from
 '@tanstack/react-query';

const queryClient = new QueryClient();
function App() {
  return <QueryClientProvider client={queryClient}><App /></QueryClientProvider>;
}

// useQuery - fetch and cache data
function UserProfile({ userId }) {
  const { data: user, isLoading, isError, error } = useQuery({
    queryKey: ['user', userId], // cache key
    queryFn: () => fetchUser(userId), // how to fetch
    staleTime: 5 * 60 * 1000, // consider fresh for 5 min
    gcTime: 10 * 60 * 1000, // keep in cache for 10 min
  });

  if (isLoading) return <Spinner />;
  if (isError) return <Error message={error.message} />;
  return <div>{user.name}</div>;
}

// useMutation - create/update/delete
function CreatePost() {
  const mutation = useMutation({
    mutationFn: (newPost) => createPost(newPost),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['posts'] }); // refetch posts
    },
  });

  return (
    <button
      onClick={() => mutation.mutate({ title: 'Hello', body: 'World' })}
      disabled={mutation.isPending}
    >
      {mutation.isPending ? 'Saving...' : 'Create Post'}
    </button>
  );
}
```

Phase 5 Projects



E-Commerce Cart

Full cart system: product listing, add to cart, quantity controls, checkout. State: Zustand for cart, TanStack Query for products. Demonstrates when each tool is appropriate.

Concepts: Zustand · TanStack Query · useQuery · useMutation · optimistic updates

PHASE 6 Performance

Memoisation, code splitting, Suspense, and the React Profiler

6.1 React.memo — Prevent Unnecessary Re-renders

```
import { memo } from 'react';

// By default, a component re-renders whenever its parent re-renders
// React.memo skips re-rendering if props haven't changed (shallow compare)

const ExpensiveList = memo(function ExpensiveList({ items, onItemClick }) {
  console.log('ExpensiveList rendered'); // only logs when items/onItemClick
  change
  return (
    <ul>
      {items.map(item => (
        <li key={item.id} onClick={() => onItemClick(item.id)}>{item.name}</li>
      ))}
    </ul>
  );
});

// IMPORTANT: useCallback is needed for stable function references
function Parent() {
  const [count, setCount] = useState(0);
  const [items, setItems] = useState([...]);

  // Without useCallback: new reference every render → memo is useless
  const handleClick = useCallback((id) => {
    setItems(prev => prev.filter(i => i.id !== id));
  }, []); // stable reference

  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>{count}</button>
      <ExpensiveList items={items} onItemClick={handleClick} />
    </>
  );
}
```

6.2 Code Splitting with lazy & Suspense

```
import { lazy, Suspense } from 'react';

// lazy() - dynamically import a component
```

```
// The bundle is only downloaded when the component is first rendered
const Dashboard = lazy(() => import('./pages/Dashboard'));
const AdminPanel = lazy(() => import('./pages/AdminPanel'));
const HeavyChart = lazy(() => import('./components/HeavyChart'));

// Suspense — show a fallback while the component loads
function App() {
  return (
    <Router>
      <Suspense fallback={<PageLoader />}>
        <Routes>
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/admin" element={<AdminPanel />} />
        </Routes>
      </Suspense>
    </Router>
  );
}

// Nested Suspense for granular loading states
function ProductPage() {
  return (
    <div>
      <ProductInfo />      {/* loads immediately */}
      <Suspense fallback={<ChartSkeleton />}>
        <HeavyChart />    {/* lazy loaded */}
      </Suspense>
    </div>
  );
}
```

Phase 6 Projects



Performance Audit

Take the e-commerce app from Phase 5. Use the React DevTools Profiler to find wasted renders. Add `React.memo`, `useCallback`, lazy loading. Measure before and after. Document what actually helped.

Concepts: [React.memo](#) · [useCallback](#) · [useMemo](#) · [lazy](#) · [Suspense](#) · [Profiler](#)

PHASE 7 Ecosystem

React Router, Next.js, TypeScript, and Testing

7.1 React Router v6

```
import { BrowserRouter, Routes, Route, Link, NavLink,
        useNavigate, useParams, useSearchParams, Outlet } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/" end>Home</NavLink>
        <NavLink to="/users">Users</NavLink>
      </nav>
      <Routes>
        <Route path="/"          element={<Home />} />
        <Route path="/users"     element={<Users />} />
        <Route path="/users/:id" element={<UserDetail />} />
        <Route path="*"          element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

// Read URL params
function UserDetail() {
  const { id } = useParams();
  const [searchParams] = useSearchParams();
  const tab = searchParams.get('tab') ?? 'profile';
  return <div>User {id} - Tab: {tab}</div>;
}

// Navigate programmatically
function LoginForm() {
  const navigate = useNavigate();
  const handleSubmit = async (e) => {
    e.preventDefault();
    await login(credentials);
    navigate('/dashboard');
  };
}
```

7.2 TypeScript with React

```
// Typed props - use interface or type
```

```

interface ButtonProps {
  label:      string;
  onClick:    () => void;
  disabled?:  boolean;
  variant?:   'primary' | 'secondary' | 'danger';
}

function Button({ label, onClick, disabled = false, variant = 'primary' }:
  ButtonProps) {
  return <button className={`btn-${variant}`} onClick={onClick}
    disabled={disabled}>{label}</button>;
}

// Generic components
interface ListProps<T> {
  items:      T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string;
}
function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return <ul>{items.map(item => <li
    key={keyExtractor(item)}>{renderItem(item)}</li>)}</ul>;
}

// Typing events
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) =>
  setValue(e.target.value);
const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
};
const handleClick = (e: React.MouseEvent<HTMLButtonElement>) => { };

// Typing useState
const [user, setUser] = useState<User | null>(null);
const [items, setItems] = useState<Item[]>([]);

// Typing refs
const inputRef = useRef<HTMLInputElement>(null);
inputRef.current?.focus(); // optional chaining because initially null

// Typing context
interface AuthContextType { user: User | null; login: (u: User) => void; }
const AuthContext = createContext<AuthContextType | null>(null);
const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used within AuthProvider');
  return ctx;
};

```

7.3 Testing with Vitest & React Testing Library

```

// npm install -D vitest @testing-library/react @testing-library/user-event jsdom

import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

```

```
import { describe, it, expect, vi } from 'vitest';

// Test a component
describe('Button', () => {
  it('calls onClick when clicked', async () => {
    const user = userEvent.setup();
    const handleClick = vi.fn();

    render(<Button label="Submit" onClick={handleClick} />);

    await user.click(screen.getByRole('button', { name: /submit/i }));
    expect(handleClick).toHaveBeenCalledTimes(1);
  });

  it('is disabled when disabled prop is true', () => {
    render(<Button label="Submit" onClick={() => {}} disabled />);
    expect(screen.getByRole('button')).toBeDisabled();
  });
});

// Test async components
describe('UserProfile', () => {
  it('shows user name after loading', async () => {
    vi.spyOn(global, 'fetch').mockResolvedValue({
      ok: true,
      json: async () => ({ name: 'Alice', email: 'alice@test.com' }),
    } as Response);

    render(<UserProfile userId="1" />);
    expect(screen.getByText(/loading/i)).toBeInTheDocument();

    await waitFor(() => expect(screen.getByText('Alice')).toBeInTheDocument());
  });
});
```

Phase 7 Capstone Projects



Full-Stack Dashboard

A full dashboard app with React Router (nested routes, protected routes), TanStack Query for all data, Zustand for UI state, TypeScript throughout, and a test suite with 80%+ coverage.

Concepts: [React Router v6](#) · [TanStack Query](#) · [Zustand](#) · [TypeScript](#) · [Vitest](#) · [RTL](#)



Next.js Blog with RSC

Rebuild the dashboard as a Next.js App Router app. Use React Server Components for static content, Client Components for interactivity. Implement ISR (Incremental Static Regeneration) for blog posts.

Concepts: [Next.js App Router](#) · [RSC](#) · [ISR](#) · [server actions](#) · [metadata](#)

You have reached the end of the React coursework.

Now build something. The best React developers build a lot of small things, then a few big ones.